

BİLGİSAYAR AĞLARI DÖNEM PROJESİ

TCP Tabanlı Mesajlaşma ve Dosya Gönderim Uygulaması

Öğrenci Adı Soyadı: Mehmet ÇELİK, Özlem CESUROĞLU

Öğrenci No: 230007028, 230007027

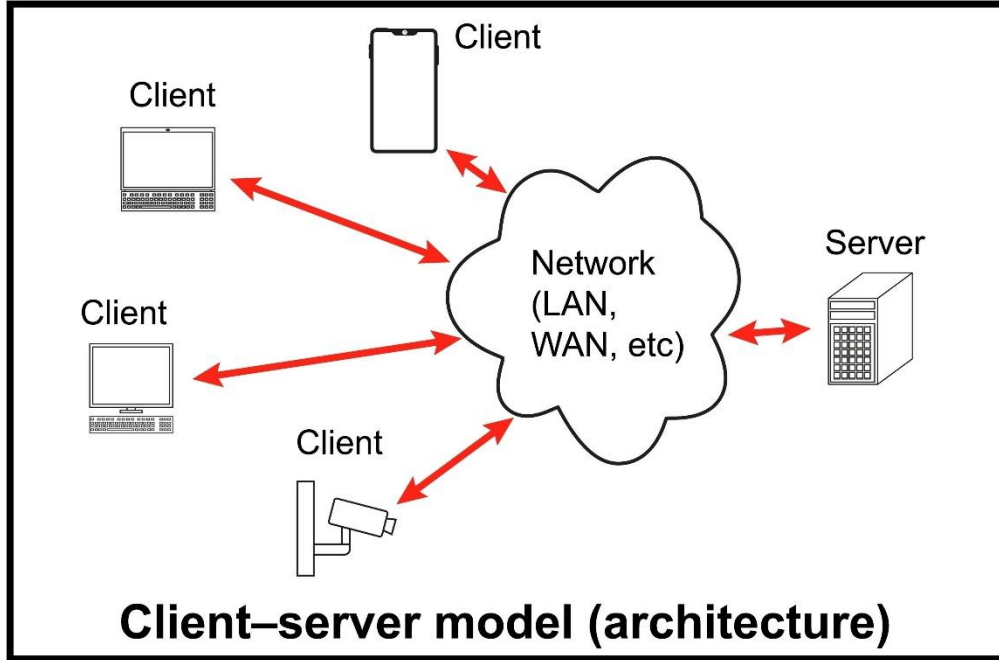
1. Projenin Amacı

Bu projenin temel amacı, TCP (Transmission Control Protocol) üzerinde çalışan, birden fazla istemcinin eşzamanlı olarak bağlanabildiği, güvenilir bir istemci-sunucu (Client-Server) haberleşme sistemi geliştirmektir .

Proje kapsamında; ağ programlamanın temelleri olan soket yönetimi, bağlantı sürekliliği, veri çerçeveleme (framing), çoklu iş parçacığı (threading) kullanımı ve veri akışı (streaming) konularının pratiğe dökülmesi hedeflenmiştir. Ayrıca, binary dosyaların (resim, pdf vb.) JSON veri formatı içerisinde bozulmadan aktarılması amaçlanmıştır.

2. Sistem Mimarisi ve Tasarım

Proje, **Merkezi Sunucu Mimarisi** kullanılarak tasarlanmıştır. Sistemde istemciler birbirleriyle doğrudan (P2P) bağlantı kurmazlar; tüm veri trafiği ve iletişim yönetimi sunucu üzerinden sağlanır .



GAZİANTEP İSLAM BİLİM VE TEKNOLOJİ ÜNİVERSİTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ
BMB501/BMB507 Bilgisayar Ağlarına Giriş
2025-2026 GÜZ DÖNEMİ PROJE

2.1. Sunucu (Server) Tasarımı

Sunucu uygulaması (server.py), belirtilen IP ve Port üzerinde (varsayılan: 12345) dinleme modunda çalışır.

Eşzamanlılık (Concurrency): Sunucuya bağlanan her yeni istemci için Python threading kütüphanesi kullanılarak ayrı bir iş parçacığı (thread) oluşturulur. Bu sayede sunucu, bir kullanıcıyla dosya transferi yaparken diğer kullanıcıların sohbet trafiğini engellemez.

Yönlendirme (Routing): Sunucu, bağlı kullanıcıların socket nesnelerini connected_users isimli bir sözlük yapısında tutar. Gelen mesajın to (kime) alanına bakarak mesajı doğru sokete yönlendirir.

2.2. İstemci (Client) Tasarımı

İstemci uygulaması (client.py), kullanıcıdan aldığı komutları sunucuya iletir ve sunucudan gelen yanıtları işler.

Dinleme Thread'i: Kullanıcı mesaj yazarken aynı anda gelen mesajları kaçırmamak için listen_for_messages fonksiyonu arka planda (Daemon Thread olarak) sürekli çalışır.

Komut Yapısı: Kullanıcı arayüzünde list, dosya:kime:yol, kabul:kimden:dosya gibi metin tabanlı komutlar işlenerek uygun JSON formatına çevrilir.

2.3. Veri Akışı ve Protokol

TCP protokolü bir veri akışı (stream) olduğu için mesaj sınırlarını belirlemek adına "Uzunluk Ön Ekli Çerçeveleme" (Length-Prefixed Framing) yöntemi kullanılmıştır .

Paket Yapısı: Her mesajın başına, o mesajın uzunluğunu belirten 4 baytlık (Big Endian formatında) bir başlık eklenmiştir.

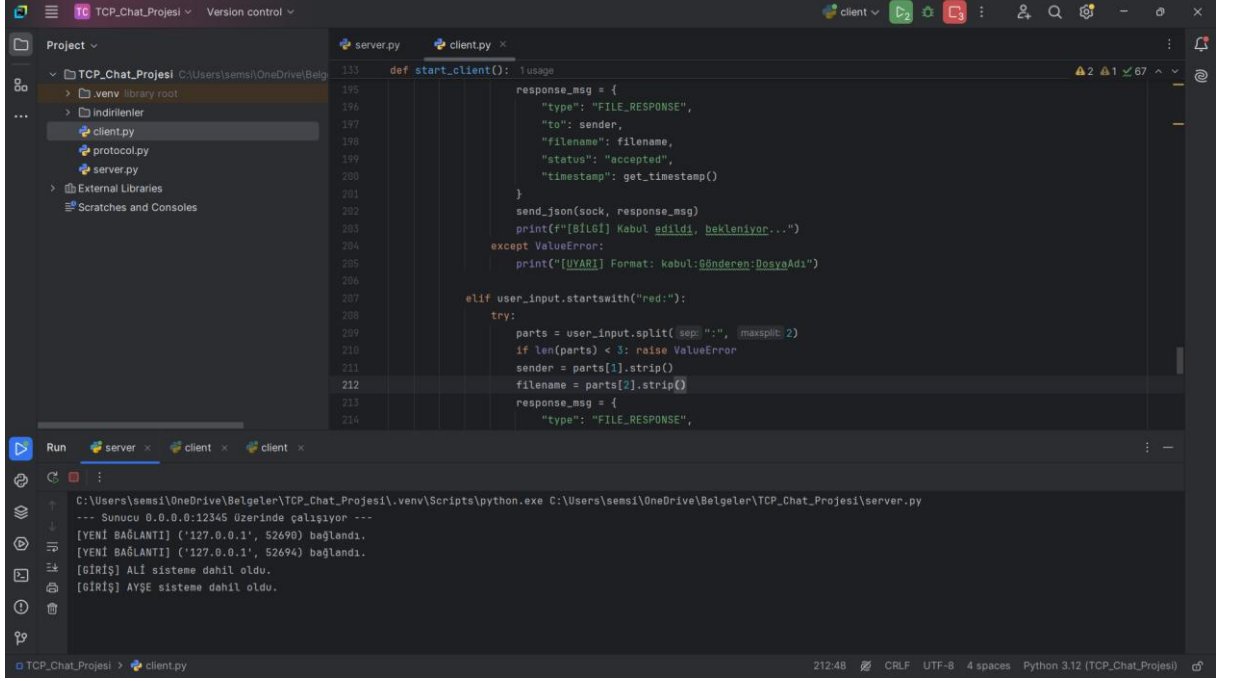
İçerik: Veri formatı olarak JSON kullanılmıştır.

Dosya Transferi: Dosyalar belleği şişirmemek için 4KB'lık küçük parçalara (Chunk) bölünür. Binary veri, JSON içinde taşınabilmesi için Base64 ile kodlanır.

GAZİANTEP İSLAM BİLİM VE TEKNOLOJİ ÜNİVERSİTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ
BMB501/BMB507 Bilgisayar Ağlarına Giriş
2025-2026 GÜZ DÖNEMİ PROJE

3. Ekran Görüntüleri

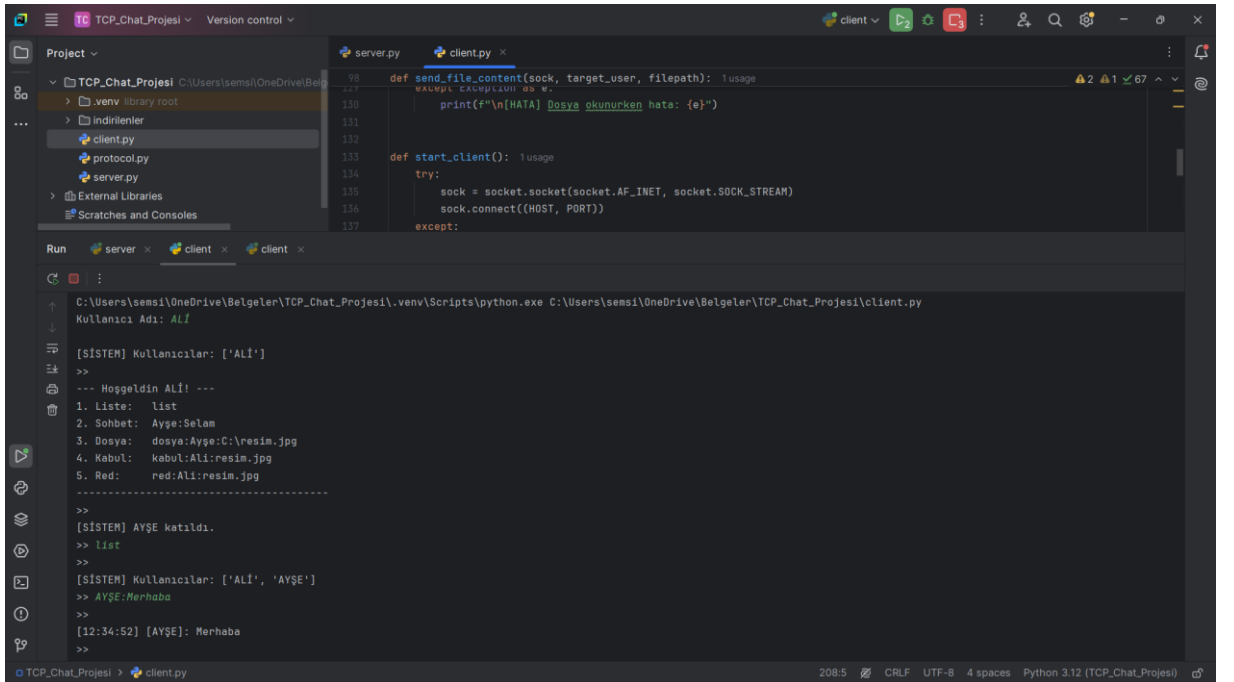
3.1. Sunucunun Başlatılması ve İstemci Girişleri



```
def start_server():  
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)  
    sock.bind((HOST, PORT))  
    sock.listen(5)  
    print(f"[BİLGİ] Sunucu {HOST}:{PORT} üzerinde çalışıyor...")  
    while True:  
        conn, addr = sock.accept()  
        print(f"[YENİ BAĞLANTI] ({addr}) bağlandı.")  
        thread = threading.Thread(target=handle_client, args=(conn, addr))  
        thread.start()  
        print(f"[GİRİŞ] {addr} sisteme dahil oldu.")  
  
def start_client():  
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)  
    sock.connect((HOST, PORT))  
    print(f"[BİLGİ] İstemci {HOST}:{PORT} ile bağlantı kuruldu.")  
    while True:  
        user_input = input("Mesaj girin (red: çıkış için): ")  
        if user_input == "red":  
            break  
        send_json(sock, {"type": "FILE_RESPONSE", "to": "sender", "filename": "dosya.jpg", "status": "accepted", "timestamp": get_timestamp()})  
        print(f"[UYARI] Format: kabul:Gönderen:DosyaAdı")  
        if user_input.startswith("red:"): try:  
            parts = user_input.split(":", maxsplit=2)  
            if len(parts) < 3: raise ValueError  
            sender = parts[1].strip()  
            filename = parts[2].strip()  
            response_msg = {"type": "FILE_RESPONSE",
```

```
C:\Users\semsi\OneDrive\Belgeler\TCP_Chat_Projesi\.venv\Scripts\python.exe C:\Users\semsi\OneDrive\Belgeler\TCP_Chat_Projesi\server.py  
--- Sunucu 0.0.0.0:12345 üzerinde çalışıyor ---  
[YENİ BAĞLANTI] ('127.0.0.1', 52690) bağlandı.  
[YENİ BAĞLANTI] ('127.0.0.1', 52694) bağlandı.  
[GİRİŞ] ALİ sisteme dahil oldu.  
[GİRİŞ] AYŞE sisteme dahil oldu.
```

3.2. Kullanıcı Listeleme ve Sohbet



```
def send_file_content(sock, target_user, filepath):  
    try:  
        with open(filepath, "rb") as f:  
            data = f.read()  
            sock.sendall(data)  
    except Exception as e:  
        print(f"[HATA] Dosya okunurken hata: {e}")  
  
def start_client():  
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)  
    sock.connect((HOST, PORT))  
    print(f"[BİLGİ] İstemci {HOST}:{PORT} ile bağlantı kuruldu.")  
    while True:  
        user_input = input("Mesaj girin (red: çıkış için): ")  
        if user_input == "red":  
            break  
        send_json(sock, {"type": "FILE_RESPONSE", "to": "sender", "filename": "dosya.jpg", "status": "accepted", "timestamp": get_timestamp()})  
        print(f"[UYARI] Format: kabul:Gönderen:DosyaAdı")  
        if user_input.startswith("red:"): try:  
            parts = user_input.split(":", maxsplit=2)  
            if len(parts) < 3: raise ValueError  
            sender = parts[1].strip()  
            filename = parts[2].strip()  
            response_msg = {"type": "FILE_RESPONSE",
```

```
C:\Users\semsi\OneDrive\Belgeler\TCP_Chat_Projesi\.venv\Scripts\python.exe C:\Users\semsi\OneDrive\Belgeler\TCP_Chat_Projesi\client.py  
Kullanıcı Adı: ALİ  
[SİSTEM] Kullanıcılar: ['ALİ']  
>>>  
--- Hoggeldin ALİ! ---  
1. Liste: List  
2. Sohbet: Ayşe:Selam  
3. Dosya: dosya:Ayşe:C:\resim.jpg  
4. Kabul: kabul:Ali:resim.jpg  
5. Red: red:Ali:resim.jpg  
-----  
>>>  
[SİSTEM] AYŞE katıldı.  
>>> list  
>>>  
[SİSTEM] Kullanıcılar: ['ALİ', 'AYŞE']  
>>> AYŞE:Merhaba  
>>>  
[12:34:52] [AYŞE]: Merhaba  
>>>
```

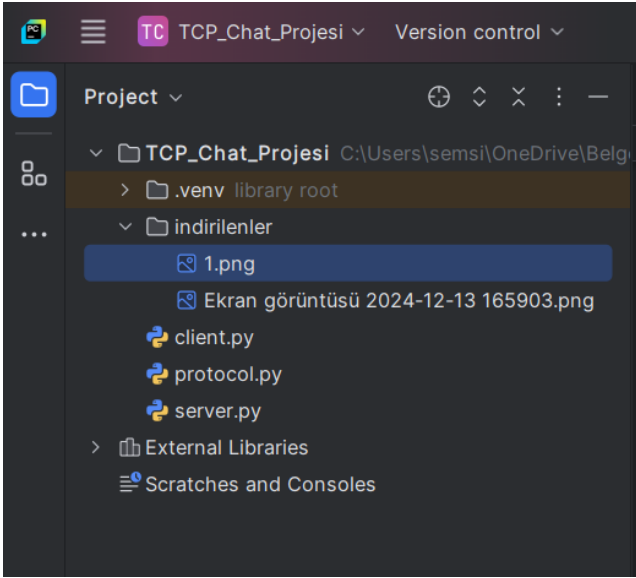
GAZİANTEP İSLAM BİLİM VE TEKNOLOJİ ÜNİVERSİTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ
BMB501/BMB507 Bilgisayar Ağlarına Giriş
2025-2026 GÜZ DÖNEMİ PROJE

3.3. Dosya Gönderme Teklifi ve Kabulü

```
>> dosya:AYŞE:C:\1.png
[BİLGİ] Teklif gönderildi...
>>
[BİLGİ] AYŞE kabul etti. Gönderim başlıyor...
>>
[BİLGİ] Transfer tamamlandı: 1.png
>>
```

```
[DOSYA] ALİ size '1.png' (2201025 bayt) göndermek istiyor.
Kabul için: kabul:ALİ:1.png
Red için: red:ALİ:1.png
>> kabul:ALİ:1.png
[BİLGİ] Kabul edildi, bekleniyor...
>>
[BAŞARILI] '1.png' indi.
>>
```

3.4. Başarılı Dosya Transferi ve İndirilenler Klasörü



```
Run server client client
C:\Users\semsi\OneDrive\Belgeler\TCP_Chat_Projesi\.venv\Scripts\python.exe C:\Users\semsi\OneDrive\Belgeler\TCP_Chat_Projesi\server.py
--- Sunucu 0.0.0.0:12345 üzerinde çalışıyor ---
[YENİ BAĞLANTI] ('127.0.0.1', 52690) bağlandı.
[YENİ BAĞLANTI] ('127.0.0.1', 52694) bağlandı.
[GİRİŞ] ALİ sisteme dahil oldu.
[GİRİŞ] AYŞE sisteme dahil oldu.
[ALİ] Gelen: {'type': 'GET_USER_LIST'}
[ALİ] Gelen: {'type': 'CHAT_MSG', 'to': 'AYŞE', 'text': 'Merhaba', 'timestamp': '2026-01-02T12:34:29'}
[İLETLİDİ] ALİ -> AYŞE (CHAT_MSG)
[AYŞE] Gelen: {'type': 'CHAT_MSG', 'to': 'ALİ', 'text': 'Merhaba', 'timestamp': '2026-01-02T12:34:52'}
[İLETLİDİ] AYŞE -> ALİ (CHAT_MSG)
[ALİ] Gelen: {'type': 'FILE_OFFER', 'to': 'AYŞE', 'filename': '1.png', 'filesize': 2201025, 'timestamp': '2026-01-02T12:36:46'}
[İLETLİDİ] ALİ -> AYŞE (FILE_OFFER)
[AYŞE] Gelen: {'type': 'FILE_RESPONSE', 'to': 'ALİ', 'filename': '1.png', 'status': 'accepted', 'timestamp': '2026-01-02T12:36:56'}
[İLETLİDİ] AYŞE -> ALİ (FILE_RESPONSE)
[ALİ] Gelen: {'type': 'FILE_DONE', 'to': 'AYŞE', 'filename': '1.png', 'timestamp': '2026-01-02T12:37:07'}
[İLETLİDİ] ALİ -> AYŞE (FILE_DONE)
```

4. Karşılaşılan Zorluklar ve Çözümler

Proje geliştirme sürecinde karşılaşılan temel teknik zorluklar ve üretilen çözümler aşağıdadır:

TCP Veri Yapışması (Sticky Packet):

Zorluk: TCP'nin mesajları birleştirerek göndermesi sonucu, arka arkaya gelen JSON verilerinin iç içe geçmesi ve kodun çökmesi sorunu yaşandı.

Çözüm: protocol.py modülü yazılarak her paketin başına 4 baytlık uzunluk bilgisi eklendi (struct kütüphanesi ile). Alıcı taraf önce bu 4 baytı okuyup paketin boyutunu öğrendi, sonra tam o boyut kadar veriyi bekledi.

Dosya Yolu Kaybı:

Zorluk: Dosya gönderme teklifi (FILE_OFFER) yapıldığında karşı tarafa sadece dosya adı gidiyordu. Alıcı kabul ettiğinde, gönderici program dosyanın diskteki tam yolunu unuttuğu için "Dosya bulunamadı" hatası alındı.

Çözüm: İstemci tarafında offered_files adında bir sözlük (dictionary) yapısı oluşturuldu. Teklif edilen dosyanın tam yolu bu hafızaya kaydedildi. Onay mesajı geldiğinde dosya yolu hafızadan çekilerek transfer başlatıldı.

Arayüzün Donması (Blocking I/O):

Zorluk: Sunucudan mesaj beklenirken (recv komutu sırasında), programın kilitlenmesi ve kullanıcının o sırada mesaj yazamaması.

Çözüm: Python threading kütüphanesi kullanılarak mesaj dinleme işlemi ana döngüden ayrıldı. Böylece kullanıcı yazı yazarken arka planda gelen mesajlar işlenebildi.